UT1 · KOTLIN FUNDAMENTALS

Block 4

Null safety and collections

What you will learn
Understand why `null` is dangerous and how Kotlin protects you from it
Use nullable types (`String?`), the safe call `?.` and the Elvis operator `?:`
Store several values in a `List` and change them with a `MutableList`
Traverse, filter and transform lists with `for`, `filter`, `map` and `find`
Use `Set` for values without duplicates and `Map` for key–value pairs

This block has two parts. The first one deals with **null**, the value that means "there is nothing here". It is the cause of the most common error in Java, the `NullPointerException`, and Kotlin has a whole set of tools to avoid it. The second part introduces **collections**: lists, sets and maps, the structures we use to store many values together.

Both parts are closely related: as you will see, many collection operations, such as searching for an element that may not exist, return `null`. And both are essential in Compose, where a screen very often shows a list of elements, or a message when some data is missing.

> **TIP · Lambdas are needed here**
> Section 10 uses lambdas such as `{ it > 5 }`. Review the **Lambda expressions** notes before that section if you need to.

Contenido

1. The problem with null 3
2. Nullable types 3
2.1 Where does null come from? 4
3. Checking for null with if 4
4. The safe call operator ?. 4
5. The Elvis operator ?: 5
5.1 Safe keyboard input 5
5.2 Elvis with return 5

6. The not-null assertion !! (avoid it).....6
7. Lists: List 6
8. Mutable lists: MutableList 7
8.1 val with a mutable list? 7
9. Traversing lists 8
10. Common operations: filter, map, find.....9
10.1 find and null safety 10
10.2 Chaining operations 10
11. Sets: Set.....10
12. Maps: Map 11
12.1 Mutable maps and traversal 11
13. Which collection should I use? 12
14. Summary: Java vs Kotlin 12
15. Exercises.....13
Exercise 1 · Safe input.....13
Exercise 2 · Predict the output 13
Exercise 3 · Create your own contact book.....13
Exercise 4 · Marks analysis 14
Exercise 5 · Unique visitors.....14
Exercise 6 · Manage a product list.....14
Challenge · Shop with stock.....14

PART A · NULL SAFETY

1. The problem with null

In Java, any object variable can contain `null`. If you call a method on it, the program crashes with a `NullPointerException`:

```
Java
String name = null;
System.out.println(name.length());   // NullPointerException at runtime
```

The worst part is that the compiler does not warn you: the error only appears when the program is running, often in front of the user. The inventor of `null`, Tony Hoare, called it his "billion-dollar mistake".

Kotlin solves this at the root: **by default, a variable cannot contain null**. If you try, the code does not compile:

```
var name: String = "Laura"
name = null          // ERROR: Null cannot be a value of a non-null type 'String'
```

This means that whenever you see a type such as `String` or `Int` in Kotlin, you can be sure it always has a real value. The error moves from **runtime** (the app crashes) to **compile time** (IntelliJ underlines it in red before you run anything).

2. Nullable types

Sometimes "no value" is a perfectly valid situation: a user without a second surname, a product without a discount, a search that finds nothing. For these cases, add a question mark `?` after the type. A **nullable type** can contain a value **or** `null`:

```
fun main() {
    val firstSurname: String = "García"
    var secondSurname: String? = null     // may be empty
    println(secondSurname)                // null

    secondSurname = "López"
    println(secondSurname)                // López
}
```

`String` and `String?` are **different types**. Because a `String?` might be `null`, Kotlin does not let you use it as if it were a normal `String`:

```
val secondSurname: String? = "López"
println(secondSurname.length)
// ERROR: Only safe (?.) or non-null asserted (!!.) calls are allowed
// on a nullable receiver of type 'String?'
```

The compiler forces you to decide what to do in case the value is `null`. The next sections show the four ways to do it.

2.1 Where does null come from?

You will rarely write `= null` yourself. Most nullable values come from functions that **may not have an answer**. You have already used some of them:

Function	Returns null when…
`"42".toIntOrNull()`	the text is not a valid number (`"abc".toIntOrNull()` is `null`)
`readlnOrNull()`	there is no more input to read
`list.find { … }`	no element meets the condition (section 10)
`map[key]`	the key does not exist in the map (section 12)
`list.firstOrNull()`, `list.maxOrNull()`	the list is empty

> **TIP · The ...OrNull convention**
> Many Kotlin functions come in two versions. `toInt()`, `first()` or `max()` **crash** if they cannot answer. Their `...OrNull` version returns `null` instead, and the compiler then makes you handle that case. Prefer the `...OrNull` versions whenever the data comes from the user or may be missing.

3. Checking for null with if

The most direct way is to check it with an `if`, as you would in Java. The interesting part is that, **inside** the `if`, Kotlin already knows the value is not `null`, so you can use it as a normal `String`. This is called a **smart cast**:

```
fun main() {
    val secondSurname: String? = "López"

    if (secondSurname != null) {
        println("Length: ${secondSurname.length}")   // smart cast to String
    } else {
        println("No second surname")
    }
}
```

IntelliJ highlights smart-cast variables with a green background, so you can see where Kotlin knows that the value is safe.

4. The safe call operator ?.

Writing an `if` for every nullable value is tiring. The **safe call operator** `?.` does it for you: if the value is not null, it calls the function; if it is null, it does nothing and the whole expression is `null`.

```
fun main() {
    val withSurname: String? = "López"
    val withoutSurname: String? = null

    println(withSurname?.length)        // 5
    println(withoutSurname?.length)     // null (no crash)
    println(withoutSurname?.uppercase()) // null
}
```

Read `a?.b` as **"if a is not null, give me b; otherwise, null"**. Notice that the result of a safe call is also nullable: `withSurname?.length` is of type `Int?`, not `Int`.

5. The Elvis operator ?:

Often we do not want a `null` as the result, but a **default value**. The **Elvis operator** `?:` (turn your head to the left and you will see why it is called that) returns the value on its left if it is not null, or the value on its right otherwise:

```
fun main() {
    val secondSurname: String? = null

    val surnameToShow = secondSurname ?: "(none)"
    println(surnameToShow)                 // (none)

    val length = secondSurname?.length ?: 0
    println(length)                        // 0
}
```

`?.` and `?:` are usually used together, as in the second example: "the length of the surname, or 0 if there is no surname". The result is a normal `Int`, not an `Int?`, because the Elvis operator guarantees there is always a value.

5.1 Safe keyboard input

In Block 1, `readln().toInt()` crashed the program if the user typed letters. Now we can fix it:

```
fun main() {
    print("How many units? ")
    val quantity = readln().toIntOrNull() ?: 1   // invalid input -> 1

    println("You ordered $quantity units")
}
```

5.2 Elvis with return

The right side of `?:` can also be a `return`. This creates a very clean **early return** (Block 3) for null values:

```
fun printDiscountedPrice(priceText: String) {
    val price = priceText.toDoubleOrNull() ?: return   // not a number: stop here
    println("Price with discount: ${price * 0.9}")
}

fun main() {
    printDiscountedPrice("50")      // Price with discount: 45.0
    printDiscountedPrice("hello")   // prints nothing
}
```

6. The not-null assertion !! (avoid it)

The operator `!!` tells the compiler: "trust me, this is not null". It converts a `String?` into a `String`, but if the value **is** null, the program crashes with a `NullPointerException`, exactly the error Kotlin was trying to protect you from.

```
val secondSurname: String? = null
println(secondSurname!!.length)   // crashes: NullPointerException
```

WATCH OUT · Treat !! as a warning sign

Every `!!` in your code is a place where the program can crash. In these exercises, you should never need it: use `if`, `?.` or `?:` instead. If IntelliJ suggests `!!` as a quick fix, it is almost always better to choose another option.

Tool	Use it when…	Example
`if (x != null)`	You need to do several things with the value	`if (name != null) { … }`
`?.`	You want to use the value only if it exists	`name?.length`
`?:`	You need a default value	`name ?: "Anonymous"`
`!!`	Practically never	—

PART B · COLLECTIONS

7. Lists: List

A **list** stores several values of the same type, **in order**, and allows repeated values. Create it with `listOf`:

```
fun main() {
    val courses = listOf("Kotlin", "Java", "Python", "Kotlin")

    println(courses)            // [Kotlin, Java, Python, Kotlin]
    println(courses.size)       // 4
    println(courses[0])         // Kotlin  (positions start at 0)
    println(courses.first())    // Kotlin
    println(courses.last())     // Kotlin
    println("Java" in courses)  // true
    println(courses.indexOf("Python"))  // 2
}
```

The type of this list is `List<String>`: a list of texts. Kotlin infers it from the elements, as always. The type between the angle brackets `< >` is the type of the elements.

> **WATCH OUT · Positions out of range**
> The valid positions of a list go from `0` to `size - 1`. `courses[10]` compiles, but crashes with an `IndexOutOfBoundsException`. If the position may not exist, use `courses.getOrNull(10)`, which returns `null` instead (and you already know how to handle it).

A `List` created with `listOf` is **read-only**: it has no functions to add, remove or change elements. This is intentional, just like using `val` instead of `var`: if a list does not need to change, it is safer to make it read-only.

8. Mutable lists: MutableList

When the list must change (a shopping cart, a to-do list), use `mutableListOf`:

```
fun main() {
    val cart = mutableListOf("Keyboard", "Mouse")

    cart.add("Monitor")           // add at the end
    cart.add(0, "Laptop")         // add at position 0
    println(cart)                 // [Laptop, Keyboard, Mouse, Monitor]

    cart[1] = "Wireless keyboard" // replace position 1
    cart.remove("Mouse")          // remove by value
    cart.removeAt(0)              // remove by position
    println(cart)                 // [Wireless keyboard, Monitor]

    println(cart.isEmpty())       // false
    cart.clear()                  // remove everything
    println(cart.size)            // 0
}
```

8.1 val with a mutable list?

Notice that `cart` is declared with `val`, and still we add and remove elements. There is no contradiction: `val` means that the **variable** always points to the **same list**; it says nothing about the **content** of the list.

```
val cart = mutableListOf("Keyboard")
cart.add("Mouse")            // OK: we change the content of the list
cart = mutableListOf()       // ERROR: we try to point to a different list
```

TIP · Empty lists need a type

With `listOf("a", "b")` Kotlin infers the type from the elements. An empty list has no elements to look at, so you must write the type: `val cart = mutableListOf<String>()`. Without `<String>`, the code does not compile.

Java

```
List<String> cart = new ArrayList<>();
cart.add("Keyboard");
String first = cart.get(0);
cart.set(0, "Mouse");
int n = cart.size();
```

Kotlin

```
val cart = mutableListOf<String>()
cart.add("Keyboard")
val first = cart[0]
cart[0] = "Mouse"
val n = cart.size
```

9. Traversing lists

A `for` loop goes through every element of a list, exactly as it did with ranges in Block 2:

```
fun main() {
    val courses = listOf("Kotlin", "Java", "Python")

    for (course in courses) {
        println("- $course")
    }

    // When you also need the position:
    for ((index, course) in courses.withIndex()) {
        println("${index + 1}. $course")
    }

    // With a lambda (Lambda expressions notes):
    courses.forEach { println(it.uppercase()) }
}
```

`withIndex()` gives each element together with its position. The syntax `(index, course)` separates both values into two variables. You can also traverse the positions directly with `for (i in courses.indices)`, which is the range `0..<size`.

WATCH OUT · Do not remove elements while traversing

Removing elements from a `MutableList` inside a `for` over that same list usually crashes the program with a `ConcurrentModificationException` (and, in some cases, silently skips elements, which is even worse). To remove all the elements that meet a condition, use `cart.removeIf { it.startsWith("M") }`, or create a new list with `filter` (next section).

10. Common operations: filter, map, find…

Lists have dozens of functions that receive a lambda. In the lambda, `it` is each element of the list. These are the ones you will use most:

```
fun main() {
    val prices = listOf(12.5, 99.0, 5.0, 45.0, 20.0)

    val cheap = prices.filter { it < 20 }          // [12.5, 5.0]
    val withTax = prices.map { it * 1.21 }   // new list: prices + 21 % tax
    val firstExpensive = prices.find { it > 40 }   // 99.0
    val overThousand = prices.find { it > 1000 }   // null
    val howManyCheap = prices.count { it < 20 }    // 2

    println("$cheap $firstExpensive $overThousand $howManyCheap")
    println(withTax.size)                          // 5
}
```

Function	Returns	Example	Result
`filter { }`	A new list with the elements that meet the condition	`listOf(1, 8, 3).filter { it > 2 }`	`[8, 3]`
`map { }`	A new list with each element transformed	`listOf(1, 2, 3).map { it * 10 }`	`[10, 20, 30]`
`find { }`	The **first** element that meets it, or `null`	`listOf(1, 8, 3).find { it > 2 }`	`8`
`count { }`	How many elements meet it	`listOf(1, 8, 3).count { it > 2 }`	`2`
`any { }` / `all { }`	Does any / do all meet it?	`listOf(1, 8, 3).all { it > 0 }`	`true`
`sum()`, `average()`	Total and mean of numbers	`listOf(1, 8, 3).sum()`	`12`
`maxOrNull()`, `minOrNull()`	Largest / smallest, or `null` if empty	`listOf(1, 8, 3).maxOrNull()`	`8`
`sorted()`, `sortedDescending()`	A new sorted list	`listOf(1, 8, 3).sorted()`	`[1, 3, 8]`

TIP · They never change the original list

`filter`, `map` and `sorted` return a **new** list; the original stays the same. That is why they also work on read-only lists. Store the result in a variable or use it directly.

10.1 find and null safety

`find` connects both parts of this block: since the element may not exist, it returns a **nullable** type. Combine it with what you learned in Part A:

```
fun main() {
    val courses = listOf("Kotlin", "Java", "Python")

    val result = courses.find { it.startsWith("S") }   // String?
    println(result ?: "No course starts with S")

    val length = courses.find { it.startsWith("J") }?.length ?: 0
    println(length)   // 4
}
```

10.2 Chaining operations

Because each operation returns a new list, you can chain them. Read the chain from top to bottom, like a recipe:

```
fun main() {
    val prices = listOf(12.5, 99.0, 5.0, 45.0, 20.0)

    val total = prices
        .filter { it >= 10 }      // [12.5, 99.0, 45.0, 20.0]
        .map { it * 0.9 }         // 10 % discount on each
        .sum()

    println("Total: ${"%.2f".format(total)}")   // Total: 158.85
}
```

11. Sets: Set

A **set** is a collection that **does not allow duplicates**. If you add a value that is already there, nothing happens. Use it when what matters is **whether** an element is present, not how many times or in which position.

```
fun main() {
    val tags = setOf("kotlin", "android", "kotlin", "compose")
    println(tags)          // [kotlin, android, compose]  (no duplicates)
    println(tags.size)     // 3

    val visitors = mutableSetOf<String>()
    println(visitors.add("ana"))   // true  (added)
    println(visitors.add("luis"))  // true
    println(visitors.add("ana"))   // false (already there)
    println(visitors)              // [ana, luis]
    println("luis" in visitors)    // true
}
```

A very common trick is converting a list into a set to remove repeated values: `listOf(3, 1, 3, 2, 1).toSet()` gives `[3, 1, 2]`. Sets do not have positions, so `tags[0]` does not exist.

12. Maps: Map

A **map** stores **key–value pairs**: each key is associated with one value, like a word and its definition in a dictionary, or a product and its price. Keys cannot be repeated; values can. Each pair is written with the word `to`:

```
fun main() {
    val prices = mapOf(
        "Kotlin Course" to 9.99,
        "Java Course" to 19.99,
        "Python Course" to 14.99
    )

    println(prices["Java Course"])      // 19.99
    println(prices["Swift Course"])     // null (the key does not exist)
    println(prices.size)                // 3
    println("Kotlin Course" in prices)  // true (checks the keys)
}
```

Notice that `prices["Swift Course"]` returns `null`: since the key may not exist, reading from a map always gives a **nullable** value (`Double?`). Use the Elvis operator for a default value: `prices["Swift Course"] ?: 0.0`.

12.1 Mutable maps and traversal

```
fun main() {
    val stock = mutableMapOf("Keyboard" to 10, "Mouse" to 25)

    stock["Monitor"] = 5        // add a new pair
    stock["Mouse"] = 20         // replace the value of an existing key
    stock.remove("Keyboard")    // remove a pair by its key

    for ((product, units) in stock) {
        println("$product: $units units")
    }

    println(stock.keys)         // [Mouse, Monitor]
    println(stock.values)       // [20, 5]
    println(stock.values.sum()) // 25
}
```

Java

```
Map<String, Integer> stock = new HashMap<>();
stock.put("Mouse", 25);
Integer units = stock.get("Mouse");
for (Map.Entry<String, Integer> e :
        stock.entrySet()) {
    System.out.println(e.getKey());
}
```

Kotlin

```
val stock = mutableMapOf<String, Int>()
stock["Mouse"] = 25
val units = stock["Mouse"]   // Int?
for ((product, units) in stock) {
    println(product)
}
```

13. Which collection should I use?

I need to…	Use	Example
Keep elements in order, possibly repeated	`List` / `MutableList`	Messages in a chat, steps of a recipe
Make sure there are no duplicates	`Set` / `MutableSet`	Tags, users who liked a post
Look up a value by a key	`Map` / `MutableMap`	Price of each product, phone of each contact

In every case, start with the read-only version (`listOf`, `setOf`, `mapOf`) and change to the mutable one only when the collection really has to change.

Jetpack Compose (preview): showing a list on screen

```
LazyColumn {
    items(courses) { course ->
        Text(course)
    }
}
```

In Compose, scrollable lists such as a contact list or a product catalogue are built from a Kotlin `List`. Everything in this block will be used directly there.

14. Summary: Java vs Kotlin

Concept	Java	Kotlin
Can a variable be null?	Any object, always	Only nullable types: `String?`
Null error detected…	At runtime (crash)	At compile time
Call if not null	`if (s != null) s.length()`	`s?.length`
Default value	`s != null ? s : "x"`	`s ?: "x"`
Read-only list	`List.of("a", "b")`	`listOf("a", "b")`
Mutable list	`new ArrayList<>()`	`mutableListOf<T>()`
Access by position	`list.get(0)`, `list.set(0, x)`	`list[0]`, `list[0] = x`
Map	`new HashMap<>()`, `put`, `get`	`mutableMapOf()`, `map[k] = v`, `map[k]`
Filter / transform	`stream().filter(…).collect(…)`	`filter { }`, `map { }`

15. Exercises

Create the files for this block in a new package, `exercises.collections` (right-click on `exercises` → **New** → **Package**). This way the functions and constants of these exercises will not clash with the ones you wrote in previous blocks.

Exercise 1 · Safe input

Write the function `readIntOrDefault(message: String, default: Int): Int`, which shows the message, reads a line and returns it as an `Int`, or the default value if the user does not type a valid number. Use `toIntOrNull()` and `?:`. Then use it to ask for the user's age (default 18) and the number of tickets (default 1), and print both values.

Exercise 2 · Predict the output

Without running it, write down what each line prints. Then run it and check.

```
fun main() {
    val a: String? = "Kotlin"
    val b: String? = null

    println(a?.length)
    println(b?.length)
    println(b?.length ?: -1)
    println(a ?: "empty")
    println(b ?: "empty")
    println("12".toIntOrNull()?.plus(1))
    println("doce".toIntOrNull()?.plus(1) ?: 0)
}
```

Exercise 3 · Create your own contact book

Write a program with a menu that manages a list of contact names (`MutableList<String>`). The menu must repeat until the user chooses "Exit":

```
1. Add contact
2. Show contacts
3. Search contact
4. Delete contact
5. Exit
```

- **Add:** asks for a name and adds it, but only if it is not already in the list.
- **Show:** shows the contacts numbered from 1, or "No contacts yet" if the list is empty.
- **Search:** asks for a text and shows the first contact that contains it (`find` and `contains`), or "Not found".
- **Delete:** asks for a name and removes it, saying whether it existed or not.

Use a `when` for the menu, and one function for each option. An invalid option (letters, numbers outside 1–5) must not crash the program.

Exercise 4 · Marks analysis

Given the list `val marks = listOf(7.5, 4.0, 9.0, 5.5, 3.0, 8.0, 6.5)`, show, using list functions and lambdas:

- The average mark, with two decimals.
- The highest and lowest marks.
- How many students passed (mark ≥ 5).
- The list of failing marks, sorted from lowest to highest.
- The list of marks rounded up by one point, without exceeding 10 (hint: `minOf(it + 1, 10.0)`).
- Whether any student got a 10.

Exercise 5 · Unique visitors

A web page records the users who visit it, in order, in this list: `val visits = listOf("ana", "luis", "ana", "marta", "luis", "ana", "pablo")`. Show how many visits there were in total, how many **different** users visited the page, and the list of users without repetitions. Then use a `Map` to show how many times each user visited the page.

Exercise 6 · Manage a product list

Use a `MutableMap<String, Double>` to store the products of a shop and their prices. Start with at least four products and write one function for each of the following operations:

1. Show all the products and their prices, formatted with two decimals.
2. Add a new product, or update its price if it already exists.
3. Remove a product, informing the user if it did not exist.
4. Show the price of a product chosen by the user, or "Product not found" (use `?:`).
5. Show the products cheaper than a given price.
6. Show the total value of the catalogue and the most expensive product (hint: `maxByOrNull { it.value }`).

Challenge · Shop with stock

Combine Exercises 3 and 6 into a small shop program with a menu. Use **two** maps: one for prices (`String` → `Double`) and another for stock (`String` → `Int`), and a `MutableList<String>` as the shopping cart.

- The user can add a product to the cart only if there is stock left; adding it reduces the stock by one.
- The cart can be shown with its total, including 21 % tax.
- When the user finishes the purchase, show a receipt with each product, the number of units (hint: count how many times it appears in the cart) and the total.
- No input of the user can crash the program. You must not use `!!` anywhere.